

Chapter 6

Assembly Language Programming

Learning Outcomes

After completion of this chapter students will know

- ❖ Procedure for making machine codes from assembly program
- ❖ Intel HEX file format
- ❖ Assembly language programming constructs
- ❖ Assembler instructions
- ❖ Writing and understanding assembly programs

6.1 Introduction

Assembly language programming is a low level programming containing assembly instructions. An assembly program consists of assembly instructions along with some assembler directives and controls.

6.2 Making Machine Code (HEX) File

Assembler is a utility that converts assembly instructions into microprocessor readable machine code usually called the HEX file. A complex application may consist of various modules and library functions that need to be combined as a single machine code file. It is the task of a **linker** to link and combine all object modules into a single program. The output of a **linker** is a relocatable executable file (**.exe**) that needs an operating system to run. For working with microprocessor boards and embedded systems, however, the user needs a machine code file with absolute addresses. This can be achieved by converting the **EXE** file to a **HEX** file. The steps for converting an assembly program into a **HEX** file are:

Assembler (ASM File)	→ OBJ File
Linker (OBJ File (s))	→ EXE File
EXE2BIN (EXE File)	→ BIN File
BIN2HEX (BIN File)	→ HEX or ABS File

6.3 Assembly Language Programming

An assembly language program consists of two types of statements, i) *instructions*, ii) *directives*. The assembly *instructions* are translated by the assembler into *machine instructions*. The directives give directions to the assembler during the assembly process, but are not translated into machine instructions.

Assembly Instruction Format

The general format of an assembly instruction is

Label: Mnemonic Operand1, Operand2 ; Comment

Label: A label is an identifier that is assigned to the address of the first byte of the instruction in which it appears. The presence of a label in an instruction is optional.

Mnemonic: 8086 instructions are called mnemonic. All assembly instructions must contain a mnemonic.

Operand: The presence of the operands depends on the instruction. Some instructions have no operands, some have one and some have two. If there are two operands, they are separated by a comma and assertion is from right to left (operand2 → operand1).

Comment: Comments are preceded by semicolon (;). Comments are optional and are put to improve program readability. The assembler ignores whatever is written after the semicolon.

6.3.2 Sample Assembly Program

Consider a simple task of averaging three numbers in memory and storing the result in memory location.

Before going to write an assembly program, it is advisable to make the proper steps (algorithm) for solving the task. The following algorithm may be used for averaging three numbers. Let the numbers to be averaged are NUMBER1, NUMBER2 and NUMBER3, and the result to be NUMBER4.

Algorithm:

1. Copy NUMBER1 to AL
2. Add NUMBER2 with AL and store result in AL
3. If the sum exceeds 8-bits, put the carry in the LSB of AH
4. Add NUMBER3 with AL and store result in AL
5. If the sum produces carry, add this carry to AH
6. Copy the divisor into BL register
7. Divide AX by BL
8. Set NUMBER4 = AX

Once the algorithm is ready, the next task is to realize the algorithm into an assembly program.

Program 6:1

```
; Averaging three 8-bit numbers in memory
; Program written by K. M. Rahman
CODE SEGMENT
    ASSUME     CS:CODE, DS:CODE
    ORG      100H;          ; Force IP to 100H
    MOV      AX, CS
    MOV      DS, AX        ; Make DS = CS
    XOR      AX, AX        ; Clear AX register
    MOV      AL, NUMBER1
    ADD      AL, NUMBER2
    ADC      AH, 0
    CLC                          ; Clear carry if any
    ADD      AL, NUMBER3
    ADC      AH, 0
    MOV      BL, 02
    DIV      BL
    MOV      NUMBER4, AX
NUMBER1 DB 9AH
NUMBER2 DB 89H
NUMBER3 DB 7FH
NUMBER4 DW ?
CODE ENDS
    END
```

In the above assembly program, the first two lines are comments. Look comments starts with semicolon (;). The assembler ignores the contents of a line after the semicolon.

The next two lines tell the assembler that the codes following are program codes.

```
CODE SEGMENT
        ASSUME     CS:CODE, DS:CODE
```

The last two lines declare end of code segment and end of the program.

```
CODE ENDS
        END
```

The line **ORG 100H** tells the assembler to force **IP= 100H**. The following lines assign data bytes (DB) to NUMBER1~NUMBER3 and word (DW) to NUMBER4. While NUMBER1~NUMBER3 are assigned specific 8-bit numbers, NUMBER4 assigns only empty space, as the result will be stored there.

```
NUMBER1    DB    9AH
NUMBER2    DB    89H
NUMBER3    DB    7FH
NUMBER4    DW    ?
```

Look at the program, there are few memory indirect addressing where the physical data is in the data segment (DS). Since the data is declared inside the code segment, it is necessary to make DS = CS, otherwise, if DS differs from CS, there will be erroneous result of the program.

In the program, the CS value is copied to AX and then AX copied to DS that ultimately makes DS=CS. Note that 8086 does not allow copy among segment registers.

6.4 Identifiers

Expressions

Constant Expressions:

An expression may be immediate operand, or data declared in memory locations. An expression may be a constant or a string. A constant is a number and may be represented as binary, octal, decimal or hexadecimal type. A suffix determines the number system (**B** → Binary, **O** → Octal, **D** or none → Decimal and **H** → Hexadecimal). For example, the following codes **moves** (copy) the decimal number 128 into **AL** register.

Example 6:1

```
MOV AL, 80H
MOV AL, 100000000B
MOV AL, 128
MOV AL,
```

Exceptions: If any hexadecimal number begins with A~Z, it should always be preceded with 0. If a number is **FFH**, it should be written as **OFFH**.

String Expressions:

An expression may be a string and are represented within single quotes ('').

Example 6:2

```
HALL        DB    'AHSANULLAH HALL'
INSTITUTION DB    'BUET'
DEPARTMENT  DB    'EEE'
```

Name

Name is an identifier that represents a constant, string constant or expression. In the above example, **HOUSE** is a name assigned to the string data.

6.5 Assembler Directives

Assembler directives are used in an assembly program to assist the assembler for efficient machine code generation. The following directives are most commonly used in assembly program.

Directive	Function	Example
ASSUME	Tells the assembler of the logical name of a segment	ASSUME CS: CODE, DS: DATA Tells the assembler that the logical name of CS is CODE and logical name of DS is DATA
SEGMENT	Indicates start of a logical segment	CODE SEGMENT
ENDS	Indicate end of a logical segment	CODE ENDS
END	Indicate end of the assembly program	CODE ENDS END
EQU	Equate	VALUE EQU 10
ORG	Originate (Set location counter)	ORG 1000H Set the location counter (IP) to 1000H
DB DW DD	Define Byte (1 Byte) Define Word (2 Bytes) Define Double Word (4 Bytes)	X1 DB 80H X2 DW 8080H X3 DD 80804040H
LABEL	Assigns current value of the location counter to an instruction	MOV BL, 80H LOOP1: DEC BL JNZ LOOP1
OFFSET	Tells the assembler to determine the offset of a named data item (variable) from the start of the segment that contains it.	MOV BX, OFFSET PRICE MOV AL, [BX] Moves contents from PRICE to AL
PTR	Assigns a specific type to a variable	PRICE DB 80H, 90H MOV AI, BYTE PTR PRICE Copies DS:[80H] to AL MOV AL, WORD PTR PRICE Copies DS:[9080H] to AL MOV AX, WORD PTR PRICE Copies DS:[9080H] to AL, and DS:[9081H] to AH

6.6 Intel HEX or ABS file format

Intel's Hex-record format allows program or data files to be encoded in a printable (ASCII) format. This allows viewing of the object file with standard tools and easy file transfer from one computer to another, or between a host and target. An individual Hex-record is a single line in a file composed of many Hex-records.

6.6.1 HEX-record content

Hex-Records are character strings made of several fields which specify the record type, record length, memory address, data, and checksum. Each byte of binary data

is encoded as a 2-character hexadecimal number: the first ASCII character representing the high-order 4 bits, and the second the low-order 4 bits of the byte.

The 6 fields which comprise a Hex-record are defined as follows:

Field		Characters	Description
1	Start code	1	An ASCII colon, ":".
2	Byte count	2	The count of the character pairs in the data field.
3	Address	4	The 2-byte address at which the data field is to be loaded into memory.
4	Type	2	00, 01, or 02.
5	Data	0-2n	From 0 to n bytes of executable code, or memory loadable data. n is normally 20 hex (32 decimal) or less.
6	Checksum	2	The least significant byte of the two's complement sum of the values represented by all the pairs of characters in the record except the start code and checksum.

Each record may be terminated with a CR/LF/NULL. Accuracy of transmission is ensured by the byte count and checksum fields.

6.6.2 HEX record types

There are three possible types of Hex-records.

00	A record containing data and the 2-byte address at which the data is to reside.
01	A termination record for a file of Hex-records. Only one termination record is allowed per file and it must be the last line of the file. There is no data field.
02	A segment base address record. This type of record is ignored by most programmers.

6.6.3 HEX-record example

Following is a typical Hex-record module consisting of four data records and a termination record.

Example 6:3

```
:10010000214601360121470136007EFE09D2190140
:100110002146017EB7C20001FF5F16002148011988
:10012000194E79234623965778239EDA3F01B2CAA7
:100130003F0156702B5E712B722B732146013421C7
:00000001FF
```

The first data record is explained as follows:

: Start code.

10 Hex 10 (decimal 16), indicating 16 data character pairs, 16 bytes of binary data, in this record.

01 Four-character 2-byte address field: hex address 0100,

00 indicates location where the following data is to be loaded.

00 Record type indicating a data record.

The next 16 character pairs are the ASCII bytes of the actual program data.

40 Checksum of the first Hex-record.

The termination record is explained as follows:

: Start code.
00 Byte count is zero, no data in termination record.
00 Four-character 2-byte address field, zeros.
00
01 Record type 01 is termination.
FF Checksum of termination record

6.6.4 Calculating the Checksum

As mentioned in the format table above, the last two characters represent a checksum of the data in the line. Since the checksum is a two-digit hexadecimal value, it may represent a value of 0 to 255, inclusive.

The checksum is calculated by summing the value of the data on the line, excluding the leading colon and checksum byte itself, and taking its two's complement. For example, the line:

:0300300002337A1E

Breaking this line into its components we have:

Record Length: 03 (3 bytes of data)

Address: 0030 (the 3 bytes will be stored at 0030, 0031, and 0032)

Record Type: 00 (normal data)

Data: 02, 33, 7A

Checksum: 1E

Taking all the data bytes above, we have to calculate the checksum based on the following hexadecimal values:

$$03 + 00 + 30 + 00 + 02 + 33 + 7A = E2$$

The two's complement of E2 is 1E which is, as you can, the checksum value.

For those unfamiliar with calculating a two's complement, it's quite simple: The two's complement of a number is the value which must be added to the number to reach the value 256 (decimal). That is to say, $E2 + 1E = 100$.

You may also calculate the two's complement by subtracting the value from 100h. In other words, $100h - E2h = 1Eh$ -- which is the checksum.

If the value in question is greater than FFh, simply take the part which is less than 100h. For example, if you want the two's complement of the value 494h, simply drop the leading "4" which leaves you with 94h. The two's complement of 94h is 6Ch.

Review Questions

1. Discuss the steps of producing absolute machine codes (hex) from an assembly program.
2. What are the components of an Intel hex record? Describe the Intel hex file format in detail.
3. Name the salient directives of an assembler that are mostly used during assembly programming.